Can we achieve multiple inheritance using default method :

Multiple inheritance is possible in java by using **default method** also inside an interface, here we need to use super keyword to access the method of a particular interface.
Before java 1.8, we have abstract methods inside an interface but now we can write method body(default method) inside an interface. Default method inside an interface we need to take super keyword with interface name(Alpha.super.m()).

Moreover a class is implementing from two OR more than two super interfaces and if the interfaces are containing **same default method name** then we have an ambiguity issue so, compulsory we need to override default method.

//Program :

```
package com.ravi.ed_using_default_method;

interface Alpha
{
    default void m()
    {
        SO.println("Alpha interface default method");
    }
}

interface Beta
{
    default void m()
    {
        SO.println("Beta interface default method");
    }
}

class Implementer implements Alpha, Beta
{
    @Override
    public void m()
    {
        Alpha.super.m();
        Beta.super.m();
        SO.println("Multiple inheritance is possible using default method");
    }
}

public class MultipleInheritanceUsingDefaulT
{
    public static void main(String[] args)
    {
        Implementer i = new Implementer();
        i.m();
    }
}
```

Why Java introduced Variable Hiding concept :

```
package com.ravi.ed_using_default_method;

interface X
{
    int a = 200;
}

abstract class A
{
    int a = 100;
}

class MyClass extends A implements X
{
    private int a = 300;
    public void show()
    {
        SO.println("a value is :"+a);
    }
}

public class NeedVariableHiding
{
    public static void main(String[] args)
    {
        new MyClass().show();
    }
}
```

Note : two default methods : Overriding is compulsory
two fields having same name : Variable Hiding is compulsory

Priority in between default and concrete method:

While working with class and interface, default method is having **low priority** than concrete method, in the same class is more powerful than interface.

```
interface A
{
}

class B
{
}

class C extends B implements A {} //Valid

class C implements A extends B {} //Invalid
```

//Program :

```
package com.ravi.ed_using_default_method;

interface AA
{
    default void m()
    {
        SO.println("Default Method of Interface AA");
    }
}

class BB
{
    public void m()
    {
        SO.println("Concrete Method of class BB");
    }
}

class CC extends BB  implements AA
{
}

public class PriorityDemo
{
    public static void main(String[] args)
    {
        CC c1 = new CC();
        c1.m();
    }
}
```

Can an interface extends a class ?

No, an interface can't extend a class, an interface can extend another interface.

In order to support **upcasting concept in Java**, all the public and non-final methods of Object class are re-declared inside an interface as an abstract method if that interface does not extend from another interface.

//Program :

```
package com.ravi.interface_member;

interface Hello
{
}

public class InterfaceMemberDemo1 {

    public static void main(String[] args)
    {
        Hello h = null;
        h.equals(null);  //Compiler added abstract method
        h.hashCode();    //Compiler added abstract method
        h.toString();    //Compiler added abstract method
    }
}
```

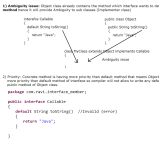
Another Program :

```
package com.ravi.interface_member;

interface Printable
{
    void print();
    public abstract String toString();
}

class Printer implements Printable
{
    @Override
    public void print()
    {
        SO.println("Printing Something");
    }

    @Override
    public String toString()
    {
        SO.println(super.toString());
        return "Java";
    }
}

public class InterfaceMemberDemo2 {

    public static void main(String[] args)
    {
        Printable p = new Printer();
        p.print();
        SO.println(p.toString());
        SO.println(p.hashCode());
    }
}
```

Can a default method of interface override/write the public method of Object class with same signature and return type.

No, a default method of an interface can't override/write the public method of Object class due to the following two reasons.

1) Ambiguity issue: Object class already contains the method which interface wants to define as a **default method** hence it will provoke Ambiguity in sub classes (Implementer class)



2) Priority: Concrete method is having more priority than default method that means Object class method is having more priority than default method of interface so compiler will not allow to write any default method as a public method of Object class.

```
package com.ravi.interface_member;

public interface Callable
{
    default String toString() //Invalid (error)

    {
        return "Java";
    }
}
```

What is a static method inside an interface ?

We can write static method (method with body) inside an interface from **java 8** version.

Example:

```
public interface Calculator
{
    static double getCuberoot(int num);//java 8 (Access Modifier is public)
}
}
```

A static method must be marked with the static keyword and contains method body.

A static method without any access modifier is assumed to be **public.**

A static method cannot be marked as **abstract OR final**

A static method is **not inherited** and cannot be accessed in a class directly. It can be accessible through interface name only.

Program that describes static method of an interface is by default public so we can access from another package.

```
package com.ravi.static_method_demo;

public interface Calculator
{
    static double add(double n, int y) //JDK 1.8
    {
        return(n+y);
    }
}

package com.ravi.ios;

import com.ravi.static_method_demo.Calculator;

public class Main {

    public static void main(String[] args)
    {
        int a = Integer.parseInt(JO.readLn("Enter the value of a :"));
        int b = Integer.parseInt(JO.readLn("Enter the value of a :"));

        double d1 = Calculator.add(a, b);
        SO.println("Sum of "+a+" and "+b+" is "+d1);
    }
}
```

Program that describes interface static methods are available to interface only.

```
interface Alpha
{
    public static void access()  //Available inside interface Alpha only
    {
        SO.println("Static Method of Alpha");
    }
}

class Beta implements Alpha
{
}

class StaticMethodDemo
{
    public static void main(String[] args)
    {
        Beta b = new Beta();
        //Beta.access();//Error
        Alpha.access();
        Alpha a=(Alpha)b;
        Alpha.access();//Only accessible through interface name only
    }
}
```

From java 9 version it is also possible to write main method inside an interface and it will be executed.

```
package com.ravi.static_method_demo;

public interface Executable
{
    public static void main(String[] args)
    {
        SO.println("Will execute");
    }
}
```

Difference between class and interface static method:

Interface Static Method:

a) Accessible using the interface name.
b) Cannot be hidden by implementing classes. (Not Available)
c) **Can be called using the interface name only.**

Class Static Method:

a) Accessible using the class name.
b) Can be hidden or overridden in subclasses by redeclaring a static method with the same signature and compatible return type.
c) Can be called using the super class, sub class name as well as sub class object.